2 Create a table Student with attributes: student_id PRIMARY KEY, name NOT NULL, email UNIQUE, age CHECK age>18

Create a table Course with attributes: course_id PRIMARY KEY, course_name NOT NULL, credits CHECK credits must be between 1 and 5 Apply constraints:

Create a table Enrollment that connects Student and Course tables, include: enroll_id PRIMARY KEY, student_id FOREIGN KEY, course_id FOREIGN KEY

Perform the following operations:

a) Insert at least 3 records into the Student table and ensure all constraints are followed.

b) Insert at least 2 records into the Course table with valid credit values.

c) Insert records into Enrollment table such that it properly references Student and Course tables.

d) Alter the Student table to- Add a new column phone , Ensure it accepts only unique values

e) Modify the Course table to change the credits constraint (e.g., allow values between 1 and 6)

f) Rename the table Student to Students.

g) Remove all records from Enrollment table using TRUNCATE

h) Delete the Course table using DROP

i) Write a query to drop a constraint from a table

```
-- Create Student table
CREATE TABLE Student (
    student_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE,
    age INT CHECK (age > 18)
);

-- Create Course table
CREATE TABLE Course (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(100) NOT NULL,
    credits INT CHECK (credits BETWEEN 1 AND 5)
);

-- Create Enrollment table
CREATE TABLE Enrollment (
    enroll_id INT PRIMARY KEY,
    student_id INT,
    course_id INT,
    FOREIGN KEY (student_id) REFERENCES Student(student_id),
    FOREIGN KEY (course_id) REFERENCES Course(course_id)
);

-------------------------------------------------
-- a) Insert records into Student table
-------------------------------------------------
```

```
INSERT INTO Student (student_id, name, email, age)
VALUES
(1, 'Rahul Sharma', 'rahul@gmail.com', 20),
(2, 'Priya Verma', 'priya@gmail.com', 22),
(3, 'Aman Gupta', 'aman@gmail.com', 19);

-------------------------------------------------
-- b) Insert records into Course table
-------------------------------------------------

INSERT INTO Course (course_id, course_name, credits)
VALUES
(101, 'Database Management System', 4),
(102, 'Operating System', 5);

-------------------------------------------------
-- c) Insert records into Enrollment table
-------------------------------------------------

INSERT INTO Enrollment (enroll_id, student_id, course_id)
VALUES
(1, 1, 101),
(2, 2, 102),
(3, 3, 101);

-------------------------------------------------
-- d) Alter Student table
-- Add phone column with UNIQUE constraint
-------------------------------------------------

ALTER TABLE Student
ADD phone VARCHAR(15) UNIQUE;

-------------------------------------------------
-- e) Modify Course table credits constraint
-- Allow values between 1 and 6
-------------------------------------------------

-- First drop old constraint (constraint name may vary)
ALTER TABLE Course
DROP CONSTRAINT credits;

-- Add new constraint
ALTER TABLE Course
ADD CONSTRAINT chk_credits
CHECK (credits BETWEEN 1 AND 6);

-------------------------------------------------
-- f) Rename Student table to Students
-------------------------------------------------
```

```
ALTER TABLE Student
RENAME TO Students;

-------------------------------------------------
-- g) Remove all records from Enrollment table
-------------------------------------------------

TRUNCATE TABLE Enrollment;

-------------------------------------------------
-- h) Delete Course table
-------------------------------------------------

DROP TABLE Course;

-------------------------------------------------
-- i) Query to drop a constraint from a table
-------------------------------------------------

ALTER TABLE table_name
DROP CONSTRAINT constraint_name;
```

3

a) Insert at least 5 records into an Employee table with attributes: emp_id, name, salary, department, city.
b) Update the salary of employees by 10% increase who belong to the 'IT' department.
c) Delete all employees whose salary is less than 20,000.
d) Write a query to display employee details where:
(i)salary is between 30,000 and 60,000 (logical operators)
(ii)department is 'HR' or 'Finance'
e) Use pattern matching to find employees whose names start with 'A'.
f) Display employee names in uppercase and calculate annual salary (salary * 12) using arithmetic and string functions.
g) Retrieve unique departments using set operators or DISTINCT.
h) Write a query to combine results of two tables (Employee and Manager) using UNION.
i) Create a new user user1 and grant the following permissions - SELECT and INSERT on Employee table.
j) Revoke INSERT permission from user1 on Employee table.
k) Start a transaction and insert a new employee record. Commit the transaction.
l) Start a transaction, update employee salary, and then ROLLBACK the changes.
m) Use SAVEPOINT after inserting records and rollback to that savepoint.

```
-------------------------------------------------
-- Create Employee table
-------------------------------------------------

CREATE TABLE Employee (
   emp_id INT PRIMARY KEY,
   name VARCHAR(100),
   salary DECIMAL(10,2),
   department VARCHAR(50),
   city VARCHAR(50)
);
```

```
-------------------------------------------------
-- a) Insert at least 5 records into Employee
-------------------------------------------------

INSERT INTO Employee (emp_id, name, salary, department, city)
VALUES
(1, 'Aman Sharma', 45000, 'IT', 'Pune'),
(2, 'Riya Mehta', 35000, 'HR', 'Mumbai'),
(3, 'Arjun Patel', 18000, 'Finance', 'Delhi'),
(4, 'Sneha Roy', 60000, 'IT', 'Bangalore'),
(5, 'Ankit Verma', 25000, 'Marketing', 'Chennai');

-------------------------------------------------
-- b) Update salary of IT employees by 10%
-------------------------------------------------

UPDATE Employee
SET salary = salary + (salary * 0.10)
WHERE department = 'IT';

-------------------------------------------------
-- c) Delete employees whose salary is less than 20,000
-------------------------------------------------

DELETE FROM Employee
WHERE salary < 20000;

-------------------------------------------------
-- d) Display employee details
-------------------------------------------------

-- (i) Salary between 30,000 and 60,000

SELECT *
FROM Employee
WHERE salary BETWEEN 30000 AND 60000;

-- (ii) Department is HR or Finance

SELECT *
FROM Employee
WHERE department = 'HR'
   OR department = 'Finance';

-------------------------------------------------
-- e) Pattern matching
-- Find employees whose names start with 'A'
-------------------------------------------------

SELECT *
FROM Employee
WHERE name LIKE 'A%';
```

```
-------------------------------------------------
-- f) Display names in uppercase and annual salary
-------------------------------------------------

SELECT
   UPPER(name) AS employee_name,
   salary,
   (salary * 12) AS annual_salary
FROM Employee;

-------------------------------------------------
-- g) Retrieve unique departments
-------------------------------------------------

SELECT DISTINCT department
FROM Employee;

-------------------------------------------------
-- h) Combine Employee and Manager tables using UNION
-------------------------------------------------

-- Create Manager table

CREATE TABLE Manager (
   emp_id INT,
   name VARCHAR(100),
   department VARCHAR(50)
);

-- Insert sample records

INSERT INTO Manager
VALUES
(101, 'Karan Singh', 'Management'),
(102, 'Neha Joshi', 'HR');

-- UNION query

SELECT name, department
FROM Employee

UNION

SELECT name, department
FROM Manager;

-------------------------------------------------
-- i) Create new user and grant permissions
-------------------------------------------------

CREATE USER user1 IDENTIFIED BY 'password123';
```

```
GRANT SELECT, INSERT
ON Employee
TO user1;

-------------------------------------------------
-- j) Revoke INSERT permission from user1
-------------------------------------------------

REVOKE INSERT
ON Employee
FROM user1;

-------------------------------------------------
-- k) Transaction with COMMIT
-------------------------------------------------

START TRANSACTION;

INSERT INTO Employee
VALUES (6, 'Vikas Rao', 40000, 'IT', 'Hyderabad');

COMMIT;

-------------------------------------------------
-- l) Transaction with ROLLBACK
-------------------------------------------------

START TRANSACTION;

UPDATE Employee
SET salary = salary + 5000
WHERE emp_id = 1;

ROLLBACK;

-------------------------------------------------
-- m) SAVEPOINT example
-------------------------------------------------

START TRANSACTION;

INSERT INTO Employee
VALUES (7, 'Pooja Sharma', 32000, 'HR', 'Pune');

SAVEPOINT sp1;

INSERT INTO Employee
VALUES (8, 'Rohan Das', 28000, 'Finance', 'Kolkata');

-- Rollback to savepoint
ROLLBACK TO sp1;
```

```
COMMIT;
```

4 Create an Employee table and insert data.

a) Write a query to find the total salary paid in each department using GROUP BY.

b) Write a query to find the average salary of employees in each department.

c) Display the maximum salary, minimum salary in each department.

d) Count the number of employees in each department using COUNT().

e) Find departments where the average salary is greater than 50,000 using HAVING.

f) Find the department with the highest total salary.

```
-------------------------------------------------
-- Create Employee table
-------------------------------------------------

CREATE TABLE Employee (
    emp_id INT PRIMARY KEY,
    name VARCHAR(100),
    salary DECIMAL(10,2),
    department VARCHAR(50),
    city VARCHAR(50)
);

-------------------------------------------------
-- Insert sample data
-------------------------------------------------

INSERT INTO Employee (emp_id, name, salary, department, city)
VALUES
(1, 'Aman Sharma', 45000, 'IT', 'Pune'),
(2, 'Riya Mehta', 55000, 'HR', 'Mumbai'),
(3, 'Arjun Patel', 70000, 'Finance', 'Delhi'),
(4, 'Sneha Roy', 60000, 'IT', 'Bangalore'),
(5, 'Ankit Verma', 30000, 'Marketing', 'Chennai'),
(6, 'Neha Joshi', 80000, 'Finance', 'Pune'),
(7, 'Karan Singh', 50000, 'HR', 'Delhi');

-------------------------------------------------
-- a) Total salary paid in each department
-------------------------------------------------

SELECT department,
       SUM(salary) AS total_salary
FROM Employee
GROUP BY department;

-------------------------------------------------
-- b) Average salary in each department
-------------------------------------------------

SELECT department,
       AVG(salary) AS average_salary
```

```
FROM Employee
GROUP BY department;

-------------------------------------------------
-- c) Maximum and Minimum salary in each department
-------------------------------------------------

SELECT department,
       MAX(salary) AS maximum_salary,
       MIN(salary) AS minimum_salary
FROM Employee
GROUP BY department;

-------------------------------------------------
-- d) Count number of employees in each department
-------------------------------------------------

SELECT department,
       COUNT(*) AS employee_count
FROM Employee
GROUP BY department;

-------------------------------------------------
-- e) Departments where average salary > 50,000
-------------------------------------------------

SELECT department,
       AVG(salary) AS average_salary
FROM Employee
GROUP BY department
HAVING AVG(salary) > 50000;

-------------------------------------------------
-- f) Department with the highest total salary
-------------------------------------------------

SELECT department,
       SUM(salary) AS total_salary
FROM Employee
GROUP BY department
ORDER BY total_salary DESC
LIMIT 1;
```

5	a) Consider the following schemes Supplier(SNO, Sname, Status, City) Parts (PNO, Pname, Color, Weight, City) Shipments(SNO,PNO,QTY) Write SQL queries for the following: i) Find shipment information (SNO, Sname, PNO, Pname, QTY) for those having quantity less than 157. ii) List SNO, Sname, PNO, Pname for those suppliers who made shipments of parts whose quantity is larger than the average quantity

iii) Find aggregate quantity of PNO 1692 of color green for which shipments made by supplier number who residing Mumbai.

b) Consider the following schemas
Emp(Emp_no, Emp_name, Dept_no)
Dept(Dept_no, Dept_name)
Address(Dept_name, Dept_location)
Write SQL queries for the following
i) Display the location of department where employee 'Ram' is working.
ii) Create a view to store total no of employees working in each department in ascending order
iii) Find the name of the department in which no employee is working

c) Consider the following relation schema
MOVIES(Mov_Id, Mov_Title,Mov_Year, Dir_Id)
DIRECTOR(Dir_Id, Dir_Name)
RATING(MOV_Id, Rev_Stars)
Write the SQL queries for the following.
i) List the title of all the movies directed by 'RAJ KAPOOR'
ii) Find the name of movies and number of stars for each movie. Sort the results on movies title and from higher stars to least stars.
iii) Assign the rating of all movies directed by 'Steven Spielberg' to 9.

```
--------------------------------------------------
-- a) Supplier, Parts, Shipments Queries
--------------------------------------------------

-- Schemas:
-- Supplier(SNO, Sname, Status, City)
-- Parts(PNO, Pname, Color, Weight, City)
-- Shipments(SNO, PNO, QTY)

--------------------------------------------------
-- a.i) Shipment information where quantity < 157
--------------------------------------------------

SELECT S.SNO,
       S.Sname,
       P.PNO,
       P.Pname,
       SH.QTY
FROM Supplier S
JOIN Shipments SH
   ON S.SNO = SH.SNO
JOIN Parts P
   ON SH.PNO = P.PNO
WHERE SH.QTY < 157;

--------------------------------------------------
-- a.ii) Suppliers who made shipments with quantity
```

```
-- greater than average quantity
-------------------------------------------------

SELECT S.SNO,
       S.Sname,
       P.PNO,
       P.Pname
FROM Supplier S
JOIN Shipments SH
   ON S.SNO = SH.SNO
JOIN Parts P
   ON SH.PNO = P.PNO
WHERE SH.QTY >
     (SELECT AVG(QTY)
      FROM Shipments);

-------------------------------------------------
-- a.iii) Aggregate quantity of part no 1692
-- having green color shipped by suppliers from Mumbai
-------------------------------------------------

SELECT SUM(SH.QTY) AS total_quantity
FROM Shipments SH
JOIN Supplier S
   ON SH.SNO = S.SNO
JOIN Parts P
   ON SH.PNO = P.PNO
WHERE P.PNO = 1692
  AND P.Color = 'Green'
  AND S.City = 'Mumbai';

-------------------------------------------------
-- b) Employee, Department, Address Queries
-------------------------------------------------

-- Schemas:
-- Emp(Emp_no, Emp_name, Dept_no)
-- Dept(Dept_no, Dept_name)
-- Address(Dept_name, Dept_location)

-------------------------------------------------
-- b.i) Display department location where Ram works
-------------------------------------------------

SELECT A.Dept_location
FROM Emp E
JOIN Dept D
   ON E.Dept_no = D.Dept_no
JOIN Address A
   ON D.Dept_name = A.Dept_name
WHERE E.Emp_name = 'Ram';
```

```
-------------------------------------------------
-- b.ii) Create a view for total employees
-- in each department in ascending order
-------------------------------------------------

CREATE VIEW Dept_Employee_Count AS
SELECT D.Dept_name,
       COUNT(E.Emp_no) AS total_employees
FROM Dept D
LEFT JOIN Emp E
    ON D.Dept_no = E.Dept_no
GROUP BY D.Dept_name
ORDER BY total_employees ASC;

-------------------------------------------------
-- b.iii) Find departments with no employees
-------------------------------------------------

SELECT D.Dept_name
FROM Dept D
LEFT JOIN Emp E
    ON D.Dept_no = E.Dept_no
WHERE E.Emp_no IS NULL;

-------------------------------------------------
-- c) Movies, Director, Rating Queries
-------------------------------------------------

-- Schemas:
-- MOVIES(Mov_Id, Mov_Title, Mov_Year, Dir_Id)
-- DIRECTOR(Dir_Id, Dir_Name)
-- RATING(Mov_Id, Rev_Stars)

-------------------------------------------------
-- c.i) Movies directed by 'RAJ KAPOOR'
-------------------------------------------------

SELECT M.Mov_Title
FROM MOVIES M
JOIN DIRECTOR D
    ON M.Dir_Id = D.Dir_Id
WHERE D.Dir_Name = 'RAJ KAPOOR';

-------------------------------------------------
-- c.ii) Movie names and ratings
-- sorted by movie title and higher stars first
-------------------------------------------------

SELECT M.Mov_Title,
       R.Rev_Stars
FROM MOVIES M
JOIN RATING R
```

```
   ON M.Mov_Id = R.Mov_Id
ORDER BY M.Mov_Title ASC,
        R.Rev_Stars DESC;

-------------------------------------------------
-- c.iii) Assign rating 9 to movies directed
-- by 'Steven Spielberg'
-------------------------------------------------

UPDATE RATING
SET Rev_Stars = 9
WHERE Mov_Id IN (
   SELECT M.Mov_Id
   FROM MOVIES M
   JOIN DIRECTOR D
      ON M.Dir_Id = D.Dir_Id
   WHERE D.Dir_Name = 'Steven Spielberg'
);
```

6 Consider the following schemes

Employee(emp_id ,emp_name, salary , dept_id)

Department(dept_id, dept_name, location)

a) Find employees who work in the IT department.

b) Find employees whose salary is greater than the average salary of all employees.

c) Find employees who work in the same department as 'John'.

d) Find the department name of the employee who has the highest salary.

e) Find employees who work in departments located in Mumbai/ Find employees who work in departments located in Mumbai using IN

f) Find employees who earn more than ANY employee in department 2.

```
-------------------------------------------------
-- Schemas:
-- Employee(emp_id, emp_name, salary, dept_id)
-- Department(dept_id, dept_name, location)
-------------------------------------------------

-------------------------------------------------
-- a) Find employees who work in the IT department
-------------------------------------------------

SELECT E.emp_id,
      E.emp_name,
      E.salary
FROM Employee E
JOIN Department D
   ON E.dept_id = D.dept_id
WHERE D.dept_name = 'IT';

-------------------------------------------------
-- b) Find employees whose salary is greater
-- than the average salary of all employees
-------------------------------------------------
```

```
SELECT emp_id,
       emp_name,
       salary
FROM Employee
WHERE salary >
      (SELECT AVG(salary)
       FROM Employee);

-------------------------------------------------
-- c) Find employees who work in the same
-- department as 'John'
-------------------------------------------------

SELECT emp_id,
       emp_name,
       salary,
       dept_id
FROM Employee
WHERE dept_id = (
   SELECT dept_id
   FROM Employee
   WHERE emp_name = 'John'
);

-------------------------------------------------
-- d) Find the department name of the employee
-- who has the highest salary
-------------------------------------------------

SELECT D.dept_name
FROM Employee E
JOIN Department D
   ON E.dept_id = D.dept_id
WHERE E.salary = (
   SELECT MAX(salary)
   FROM Employee
);

-------------------------------------------------
-- e) Find employees who work in departments
-- located in Mumbai using IN
-------------------------------------------------

SELECT emp_id,
       emp_name,
       salary,
       dept_id
FROM Employee
WHERE dept_id IN (
   SELECT dept_id
```

```
    FROM Department
    WHERE location = 'Mumbai'
);

-------------------------------------------------
-- f) Find employees who earn more than ANY
-- employee in department 2
-------------------------------------------------

SELECT emp_id,
       emp_name,
       salary
FROM Employee
WHERE salary > ANY (
    SELECT salary
    FROM Employee
    WHERE dept_id = 2
);
```

7 a) Consider the following table:
Stud_Marks(Roll_no,name, total_marks,category)
Write a Stored Procedure using cursor for the categorization of student. If marks scored by student in examination is <=1500 and marks>=990 then student will be placed in distinction category if marks scored are between 989 and 900 category is first class, if marks 899 and 825 category is Higher Second Class. Update the category values accordingly in the table.

b) Consider table: Areaofcircle(radius,area)
Write a Stored procedure using cursor for calculation of area of circle using values of radius from areaofcircle table & update the area value in the table accordingly.

```
-------------------------------------------------
-- a) Stored Procedure using Cursor for
-- Student Categorization
-------------------------------------------------

-- Table:
-- Stud_Marks(Roll_no, name, total_marks, category)

CREATE TABLE Stud_Marks (
    Roll_no INT PRIMARY KEY,
    name VARCHAR(100),
    total_marks INT,
    category VARCHAR(50)
);

-------------------------------------------------
-- Sample Data
-------------------------------------------------

INSERT INTO Stud_Marks
VALUES
(1, 'Amit', 1200, NULL),
```

```
(2, 'Riya', 950, NULL),
(3, 'Karan', 850, NULL),
(4, 'Neha', 780, NULL);

-------------------------------------------------
-- Stored Procedure
-------------------------------------------------

DELIMITER $$

CREATE PROCEDURE Categorize_Students()
BEGIN

    DECLARE done INT DEFAULT 0;
    DECLARE r_no INT;
    DECLARE marks INT;
    DECLARE cat VARCHAR(50);

    -- Cursor Declaration
    DECLARE stud_cursor CURSOR FOR
    SELECT Roll_no, total_marks
    FROM Stud_Marks;

    -- Handler Declaration
    DECLARE CONTINUE HANDLER
    FOR NOT FOUND SET done = 1;

    OPEN stud_cursor;

    read_loop: LOOP

        FETCH stud_cursor INTO r_no, marks;

        IF done = 1 THEN
            LEAVE read_loop;
        END IF;

        -- Categorization Logic
        IF marks >= 990 AND marks <= 1500 THEN
            SET cat = 'Distinction';

        ELSEIF marks >= 900 AND marks <= 989 THEN
            SET cat = 'First Class';

        ELSEIF marks >= 825 AND marks <= 899 THEN
            SET cat = 'Higher Second Class';

        ELSE
            SET cat = 'No Category';
        END IF;

        -- Update Category
```

```
        UPDATE Stud_Marks
        SET category = cat
        WHERE Roll_no = r_no;

    END LOOP;

    CLOSE stud_cursor;

END$$

DELIMITER ;

-------------------------------------------------
-- Execute Procedure
-------------------------------------------------

CALL Categorize_Students();

-------------------------------------------------
-- View Updated Table
-------------------------------------------------

SELECT * FROM Stud_Marks;




-------------------------------------------------
-- b) Stored Procedure using Cursor for
-- Area of Circle Calculation
-------------------------------------------------

-- Table:
-- Areaofcircle(radius, area)

CREATE TABLE Areaofcircle (
    radius DECIMAL(10,2),
    area DECIMAL(10,2)
);

-------------------------------------------------
-- Sample Data
-------------------------------------------------

INSERT INTO Areaofcircle
VALUES
(2, NULL),
(4, NULL),
(6, NULL);

-------------------------------------------------
```

```
-- Stored Procedure
-------------------------------------------------

DELIMITER $$

CREATE PROCEDURE Calculate_Area()
BEGIN

   DECLARE done INT DEFAULT 0;
   DECLARE r DECIMAL(10,2);
   DECLARE a DECIMAL(10,2);

   -- Cursor Declaration
   DECLARE area_cursor CURSOR FOR
   SELECT radius
   FROM Areaofcircle;

   -- Handler Declaration
   DECLARE CONTINUE HANDLER
   FOR NOT FOUND SET done = 1;

   OPEN area_cursor;

   area_loop: LOOP

      FETCH area_cursor INTO r;

      IF done = 1 THEN
         LEAVE area_loop;
      END IF;

      -- Area Calculation
      SET a = 3.14 * r * r;

      -- Update Area
      UPDATE Areaofcircle
      SET area = a
      WHERE radius = r;

   END LOOP;

   CLOSE area_cursor;

END$$

DELIMITER ;

-------------------------------------------------
-- Execute Procedure
-------------------------------------------------

CALL Calculate_Area();
```

```
-------------------------------------------------
-- View Updated Table
-------------------------------------------------

SELECT * FROM Areaofcircle;
```

8 a) Consider following schema
Student_fee_details (rollno, name, fee_deposited, date)
Write a trigger to preserve old values of student fee details before updating in the table

b) Consider following schema
Library(book_id, book_name, author, price)
Library_Audit (audit_id, book_id, book_name, author, price , action_type, action_date)
Write a database trigger on Library table. The System should keep track of the records that are being updated or deleted. The old value of updated or deleted records should be added in Library_Audit table

```
-------------------------------------------------
-- a) Trigger to preserve old values of
-- Student_fee_details before UPDATE
-------------------------------------------------

-- Main Table

CREATE TABLE Student_fee_details (
    rollno INT PRIMARY KEY,
    name VARCHAR(100),
    fee_deposited DECIMAL(10,2),
    date DATE
);

-------------------------------------------------
-- Backup Table to store old values
-------------------------------------------------

CREATE TABLE Student_fee_details_backup (
    rollno INT,
    name VARCHAR(100),
    fee_deposited DECIMAL(10,2),
    date DATE,
    updated_on TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-------------------------------------------------
-- Trigger Creation
-------------------------------------------------

DELIMITER $$

CREATE TRIGGER trg_student_fee_backup
BEFORE UPDATE
ON Student_fee_details
```

```
FOR EACH ROW
BEGIN

    INSERT INTO Student_fee_details_backup
    (rollno, name, fee_deposited, date)
    VALUES
    (OLD.rollno, OLD.name, OLD.fee_deposited, OLD.date);

END$$

DELIMITER ;

-------------------------------------------------
-- Example UPDATE
-------------------------------------------------

UPDATE Student_fee_details
SET fee_deposited = 25000
WHERE rollno = 1;

-------------------------------------------------
-- View Backup Records
-------------------------------------------------

SELECT * FROM Student_fee_details_backup;




-------------------------------------------------
-- b) Trigger on Library table for
-- UPDATE and DELETE audit tracking
-------------------------------------------------

-- Main Library Table

CREATE TABLE Library (
    book_id INT PRIMARY KEY,
    book_name VARCHAR(100),
    author VARCHAR(100),
    price DECIMAL(10,2)
);

-------------------------------------------------
-- Audit Table
-------------------------------------------------

CREATE TABLE Library_Audit (
    audit_id INT AUTO_INCREMENT PRIMARY KEY,
    book_id INT,
```

```
    book_name VARCHAR(100),
    author VARCHAR(100),
    price DECIMAL(10,2),
    action_type VARCHAR(20),
    action_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-------------------------------------------------
-- Trigger for UPDATE operation
-------------------------------------------------

DELIMITER $$

CREATE TRIGGER trg_library_update
BEFORE UPDATE
ON Library
FOR EACH ROW
BEGIN

    INSERT INTO Library_Audit
    (book_id, book_name, author, price, action_type)
    VALUES
    (OLD.book_id, OLD.book_name, OLD.author,
     OLD.price, 'UPDATE');

END$$

DELIMITER ;

-------------------------------------------------
-- Trigger for DELETE operation
-------------------------------------------------

DELIMITER $$

CREATE TRIGGER trg_library_delete
BEFORE DELETE
ON Library
FOR EACH ROW
BEGIN

    INSERT INTO Library_Audit
    (book_id, book_name, author, price, action_type)
    VALUES
    (OLD.book_id, OLD.book_name, OLD.author,
     OLD.price, 'DELETE');

END$$

DELIMITER ;
```

```
-------------------------------------------------
-- Example UPDATE
-------------------------------------------------

UPDATE Library
SET price = 550
WHERE book_id = 101;

-------------------------------------------------
-- Example DELETE
-------------------------------------------------

DELETE FROM Library
WHERE book_id = 102;

-------------------------------------------------
-- View Audit Records
-------------------------------------------------

SELECT * FROM Library_Audit;
```

9 Create a collection students and insert at least 5 documents with fields:_id, name, age, course, marks

a) Insert multiple documents into the students collection using a single query.

b) Retrieve all documents from the students collection.

c) Find students who have marks greater than 70.

d) Display students whose course is 'Computer Science' AND marks > 75 (use logical operators).

e) Find students whose age is less than 20 OR marks greater than 80.

f) Update the marks of a student whose name is 'Amit'.

g) Delete all students whose marks are less than 40.

h) Delete a student whose name is 'Sneha'.

```
// Create collection: students
// Insert multiple documents

db.students.insertMany([
  {
    _id: 1,
    name: "Amit",
    age: 19,
    course: "Computer Science",
    marks: 85
  },
  {
    _id: 2,
    name: "Sneha",
    age: 21,
    course: "Electronics",
    marks: 72
  },
  {
    _id: 3,
    name: "Rahul",
```

```
        age: 20,
        course: "Computer Science",
        marks: 65
    },
    {
        _id: 4,
        name: "Priya",
        age: 18,
        course: "Mechanical",
        marks: 90
    },
    {
        _id: 5,
        name: "Karan",
        age: 22,
        course: "Civil",
        marks: 35
    }
]);

-------------------------------------------------
// a) Insert multiple documents using single query
-------------------------------------------------

// Already done using insertMany()

-------------------------------------------------
// b) Retrieve all documents
-------------------------------------------------

db.students.find();

-------------------------------------------------
// c) Find students with marks greater than 70
-------------------------------------------------

db.students.find(
    { marks: { $gt: 70 } }
);

-------------------------------------------------
// d) Students whose course is 'Computer Science'
-- AND marks > 75
-------------------------------------------------

db.students.find(
    {
        $and: [
            { course: "Computer Science" },
            { marks: { $gt: 75 } }
        ]
    }
```

```
);

-------------------------------------------------
// e) Students whose age < 20
-- OR marks > 80
-------------------------------------------------

db.students.find(
   {
      $or: [
         { age: { $lt: 20 } },
         { marks: { $gt: 80 } }
      ]
   }
);

-------------------------------------------------
// f) Update marks of student named 'Amit'
-------------------------------------------------

db.students.updateOne(
   { name: "Amit" },
   { $set: { marks: 95 } }
);

-------------------------------------------------
// g) Delete students whose marks < 40
-------------------------------------------------

db.students.deleteMany(
   { marks: { $lt: 40 } }
);

-------------------------------------------------
// h) Delete student whose name is 'Sneha'
-------------------------------------------------

db.students.deleteOne(
   { name: "Sneha" }
);
```

10 Create a collection students with fields:_id, name, department, marks, city.
a) Write an aggregation query to calculate total marks for each department.
b) Find the average marks of students in each department using aggregation.
c) Display the maximum marks scored in each department.
d) Count the number of students in each city using aggregation.
e) Create an index on the name field in students collection.
f) Drop an index from the collection.

```
-------------------------------------------------
// Create students collection and insert documents
-------------------------------------------------
```

```
db.students.insertMany([
    {
        _id: 1,
        name: "Amit",
        department: "Computer Science",
        marks: 85,
        city: "Pune"
    },
    {
        _id: 2,
        name: "Sneha",
        department: "Electronics",
        marks: 72,
        city: "Mumbai"
    },
    {
        _id: 3,
        name: "Rahul",
        department: "Computer Science",
        marks: 90,
        city: "Pune"
    },
    {
        _id: 4,
        name: "Priya",
        department: "Mechanical",
        marks: 65,
        city: "Delhi"
    },
    {
        _id: 5,
        name: "Karan",
        department: "Electronics",
        marks: 78,
        city: "Mumbai"
    }
]);

-------------------------------------------------
// a) Calculate total marks for each department
-------------------------------------------------

db.students.aggregate([
    {
        $group: {
            _id: "$department",
            total_marks: { $sum: "$marks" }
        }
    }
]);

-------------------------------------------------
// b) Find average marks in each department
-------------------------------------------------

db.students.aggregate([
```

```
    {
        $group: {
            _id: "$department",
            average_marks: { $avg: "$marks" }
        }
    }
]);

-------------------------------------------------
// c) Display maximum marks in each department
-------------------------------------------------

db.students.aggregate([
    {
        $group: {
            _id: "$department",
            maximum_marks: { $max: "$marks" }
        }
    }
]);

-------------------------------------------------
// d) Count number of students in each city
-------------------------------------------------

db.students.aggregate([
    {
        $group: {
            _id: "$city",
            student_count: { $sum: 1 }
        }
    }
]);

-------------------------------------------------
// e) Create index on name field
-------------------------------------------------

db.students.createIndex(
    { name: 1 }
);

-------------------------------------------------
// f) Drop index from collection
-------------------------------------------------

db.students.dropIndex(
    { name: 1 }
);
```